MISTY OPERATIONS

Production Deployment and Environment Manual

A practical, end-to-end guide for local development, first VPS launch, PostgreSQL backups and replication, releases, recovery, and routine operations.

System: misty-server
Public API: https://mistysys.com/api
Edition: 1.0 | Prepared 2026-07-30
Classification: Internal operations

> The goal
> Make production boring. Follow the gates in order, preserve one tested artifact, back up every data plane, and rehearse recovery before the first emergency.

How to use this manual

This is an operator runbook, not a theoretical architecture document. The first deployment path is intentionally conservative: one VPS, host Nginx, Docker Compose, an API container, and PostgreSQL on a private Docker network. After that is stable, add an off-host backup and then a separate PostgreSQL standby.

> **Do not do everything in one night**
> Launch the single-node system first. Prove backups and restoration next. Add replication only after the primary is stable and monitored. Replication improves availability; it does not replace backups.

Assumptions

- The production host is Ubuntu 24.04 LTS or a close Debian-family equivalent.
- Nginx already terminates HTTPS for mistysys.com and forwards /api to port 8081.
- The server image listens on port 8080 inside its container and is published only on 127.0.0.1:8081.
- Cloudflare R2 stores binary objects; PostgreSQL stores application metadata; Cloudflare Durable Objects store journal collaboration state.
- Commands are examples. Replace values in angle brackets and pin real image digests before execution.

Document conventions

Marker	Meaning
[ ]	A required operator checkpoint.
Danger	Data loss, security exposure, or prolonged outage is possible.
Gate	Do not continue until the stated evidence is available.
Example	Copy-ready structure with placeholders that must be replaced.

Table of contents

1. The Misty production mental model

Production is a chain of trust and a set of independent data planes. A healthy API process is not enough: Nginx, TLS, PostgreSQL, object storage, collaboration state, Stripe webhooks, and Cloudflare Worker connectivity must all agree.

1.1 Request path

```
Internet
  |
  | HTTPS :443
  v
Nginx on VPS
  |
  | http://127.0.0.1:8081  (the /api prefix is preserved)
  v
misty-server container :8080
  |
  +-- private Docker network --> PostgreSQL :5432
  +-- HTTPS ------------------> Cloudflare R2
  +-- HTTPS ------------------> Stripe
  +-- HTTPS ------------------> provider integrations

Cloudflare Journal Worker
  |
  +-- HTTPS --> https://mistysys.com/api
  +-- Durable Objects --> live collaboration state
```

1.2 The four data planes

Plane	What lives there	Protection required
PostgreSQL	Users, accounts, billing metadata, spaces, library metadata, workflows, agents, sessions.	Logical backups, PITR or base backups, restore drills, optional standby.
Cloudflare R2	Uploaded files, renditions, previews, and binary assets.	Separate inventory and off-provider or versioned copy. Database backups do not include these objects.
Durable Objects	Live journal collaboration state and queues.	Cloudflare SQLite Durable Object PITR procedures and documented recovery ownership.
Configuration	Environment variables, signing keys, Stripe secrets, Cloudflare tokens, Nginx and Compose files.	Encrypted secret store or root-only files, off-host encrypted recovery copy, rotation procedure.

A backup is complete only when all data planes are covered

A PostgreSQL dump cannot restore R2 objects. R2 durability cannot undo every accidental or authorized deletion. Durable Object state has its own recovery system.

1.3 Production services

- Always running: Nginx, Docker Engine, misty-server, PostgreSQL.
- One-shot during deployment: database migrations and database permission reconciliation.
- External: Stripe, Cloudflare R2, Cloudflare Journal Worker and Durable Objects, email and model providers.
- Development only: Stripe CLI listener, trycloudflare tunnel, dev-init, and automatic dev Worker deployment.

2. Environment strategy

2.1 Environment matrix

Concern	Development	Staging	Production
Purpose	Fast local iteration.	Release rehearsal with production shape.	Real customer traffic and durable data.
API address	localhost:8081.	Private hostname or restricted HTTPS.	https://mistysys.com/api.
Database	Disposable local pgvector volume.	Isolated non-production database.	Dedicated durable primary; optional standby.
Stripe	Test mode plus stripe listen.	Test mode, registered HTTPS endpoint.	Live mode, registered HTTPS endpoint.
Cloudflare	Dev Worker and optional tunnel.	Separate Worker, R2 bucket, and secrets.	Production Worker, bucket, and secrets.
Secrets	Ignored .env.	Separate restricted secret set.	Root-only file or secret manager; never copied to dev.
Data	Synthetic or scrubbed snapshot.	Synthetic or scrubbed snapshot.	Customer data.
Deploy	docker compose up --build.	Pinned image; production-shaped Compose.	Exact tested image by immutable digest.

2.2 Non-negotiable boundaries

- Never point a local or staging API at the production database.
- Never reuse production Stripe, OAuth, Cloudflare, email, encryption, or signing secrets in development.
- Never expose PostgreSQL on a public interface. Use a private network, SSH tunnel, or VPN.
- Never deploy a mutable tag such as latest without recording and pinning its resolved digest.
- Never use production customer data in development until it has been scrubbed and the result validated.

2.3 Recommended naming

```
Environment:     development | staging | production
Compose project: misty-dev   | misty-stage | misty-prod
Database:        misty_dev    | misty_stage | misty
DB app role:     misty_app_dev| misty_app_stage | misty_app
R2 bucket:       misty-dev    | misty-stage | misty
Worker:          journal-collab-dev | journal-collab-stage | journal-collab
```

3. Local development operations

3.1 What the canonical development stack does

The repository docker-compose.yml is a development orchestrator. It creates PostgreSQL, runs migrations and role grants, starts a Stripe CLI listener, starts the API on host port 8081, optionally deploys the development Cloudflare Worker, and opens a temporary tunnel.

3.2 Start and stop

```
cd ~/misty-org/misty-server

# Rebuild the API image from current source, then start the stack.
docker compose up --build

# Start in the background.
docker compose up --build -d

# Reuse the existing image when source or Dockerfile did not change.
docker compose up -d

# Follow logs.
docker compose logs -f api postgres stripe

# Stop containers but keep database data.
docker compose down

# DANGER: stop and delete named volumes, including the local database.
docker compose down --volumes
```

> **What --build means**
> It tells Compose to build images before starting containers. Use it after application source, go.mod, or Dockerfile changes. It does not automatically delete database volumes.

3.3 Development health checks

```
docker compose ps
curl --fail --show-error http://127.0.0.1:8081/health
docker compose logs --tail=200 api
docker compose exec postgres pg_isready -U postgres
```

3.4 Why the server no longer loads .env

Docker Compose is responsible for reading the local .env and injecting selected values. The Go process reads its environment only. This removes duplicate configuration behavior and makes container and production startup consistent.

3.5 Common local failures

Symptom	Likely cause	Action
API health is 503.	Database not ready, wrong credentials, migration mismatch, or dependent service validation failed.	Inspect API and Postgres logs; confirm pg_isready; run migrations; verify environment.
Stripe is ready but events fail.	Wrong forwarding path or transient webhook secret not injected.	Confirm forwarding URL is http://api:8081/stripe/webhook and recreate Stripe/API containers.
Port already allocated.	A local process or prior stack owns 8081 or 5435.	Inspect the owning process; stop it or change the development host-port override.
No .env message.	Old binary/image still contains the former loader.	Rebuild the API image and recreate the container.
Worker cannot reach API.	Temporary tunnel changed or Worker deploy did not complete.	Restart dev tunnel/deploy services; never copy this design to production.

4. Configuration and secret management

4.1 Production configuration contract

Set MISTY_ENVIRONMENT=production to activate production validation. The application rejects missing storage, database, public URL, encryption, and Stripe inputs; it also requires HTTPS public and redirect URLs and distinct Stripe price IDs.

Group	Required production names
Core	MISTY_ENVIRONMENT, PORT, MISTY_PUBLIC_API_URL, MISTY_ALLOWED_ORIGINS, TRUST_PROXY_HEADERS
Database	DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME
R2	R2_ENDPOINT, R2_BUCKET, R2_ACCESS_KEY, R2_SECRET_KEY
Security	SPACE_LINK_ENCRYPTION_KEY
Stripe	STRIPE_SECRET_KEY, STRIPE_WEBHOOK_SECRET, four STRIPE_PRICE_* variables, STRIPE_CHECKOUT_SUCCESS_URL, STRIPE_CHECKOUT_CANCEL_URL, STRIPE_PORTAL_RETURN_URL
Stripe routing	STRIPE_WEBHOOK_PATH=/api/stripe/webhook for the current Nginx public route.

4.2 Production baseline

```
MISTY_ENVIRONMENT=production
PORT=8080
MISTY_PUBLIC_API_URL=https://mistysys.com/api
MISTY_ALLOWED_ORIGINS=https://mistysys.com
TRUST_PROXY_HEADERS=true
STRIPE_WEBHOOK_PATH=/api/stripe/webhook

DB_HOST=postgres
DB_PORT=5432
DB_NAME=misty
DB_USER=misty_app
DB_PASSWORD=<long-random-application-password>
```

> **Proxy trust is safe only with a private backend**
> The server should accept forwarded headers because Nginx terminates TLS. Keep the published API port bound to 127.0.0.1 and do not expose it directly to the Internet.

4.3 Secret file

```
sudo install -d -m 0700 -o root -g root /etc/misty-server
sudo install -m 0600 -o root -g root production.env /etc/misty-server/production.env
sudo ls -l /etc/misty-server/production.env
```

- Do not commit production.env, print it in logs, paste it into tickets, or include it in the image.
- Keep an encrypted recovery copy in a separate account or password vault.
- Use separate owner and application database passwords.
- Scope Cloudflare tokens to only required resources and operations.
- Rotate after suspected exposure and on an established schedule; record the date and owner.

4.4 Rotation sequence

1. Create the replacement secret without revoking the old one.

2. Update the root-only secret source and validate the rendered Compose configuration.
3. Recreate only affected services and verify health and external behavior.
4. Revoke the old secret and verify again.
5. Record who rotated it, when, why, and which systems were checked.

5. Preproduction decisions

5.1 Choose recovery objectives

You cannot design backups or replication without deciding how much data loss and downtime are acceptable. Start with explicit values and refine them after observing real usage.

Term	Question	Practical first target
RPO	How much committed data may be lost?	24 hours with nightly dumps; reduce to minutes with WAL archiving.
RTO	How long may restoration take?	2-4 hours for a rehearsed single-node restore.
Retention	How long must historical recovery points remain?	14 daily, 8 weekly, and 12 monthly logical backups.
Availability	Must one host failure be survivable quickly?	Add a separate standby only after backup restore is proven.

5.2 Capacity baseline

- Leave at least 30 percent free disk space for database growth, image pulls, temporary dumps, and WAL.
- Use SSD-backed storage and monitor latency, IOPS, inode use, and filesystem fullness.
- Size memory so PostgreSQL and the API do not regularly swap.
- Choose a VPS with tested snapshot behavior, but do not treat provider snapshots as the only backup.
- Budget outbound bandwidth for off-host backups and replica traffic.

5.3 Ownership

Area	Named owner must know
Deployments	How to validate, release, roll back, and freeze changes.
Database	How to migrate, back up, restore, monitor, and promote a standby.
Security	Where secrets live, how to rotate them, and how to respond to compromise.
Cloudflare	Worker deployments, Durable Object recovery, R2 inventories, and token scope.
Billing	Stripe endpoint health, delivery retries, signing secret rotation, and reconciliation.

6. Bootstrap the first VPS

> Safety gate
> Keep the original SSH session open while changing SSH or firewall settings. Test a second login before disabling password or root login. Provider console access must work.

6.1 Create an operator account

```
adduser mistyops
usermod -aG sudo mistyops
install -d -m 0700 -o mistyops -g mistyops /home/mistyops/.ssh
install -m 0600 -o mistyops -g mistyops authorized_keys /home/mistyops/.ssh/authorized_keys
```

6.2 Patch and set time

```
sudo apt update
sudo apt full-upgrade
sudo timedatectl set-timezone UTC
timedatectl status
```

6.3 Network policy

- Allow inbound TCP 22 from trusted administrator addresses where practical.
- Allow inbound TCP 80 and 443 from the Internet.
- Do not allow inbound 5432 or 8081 from the Internet.
- Remember that Docker-published ports can bypass some host firewall expectations. Bind API to loopback and use the Docker DOCKER-USER chain or provider firewall as defense in depth.
- Use a private VPN such as WireGuard or Tailscale for replication traffic between hosts.

6.4 Install Docker from the official repository

Follow the current official Docker Engine Ubuntu instructions rather than copying an old one-line installer. Install Docker Engine, the Compose plugin, and Buildx; then enable Docker at boot.

```
sudo systemctl enable --now docker
sudo docker version
sudo docker compose version
sudo docker run --rm hello-world
```

> Docker group warning
> Membership in the docker group grants control comparable to root. Limit it to trusted administrators. Prefer sudo docker or a carefully managed deployment account.

6.5 Install Nginx and Certbot

```
sudo apt install nginx certbot python3-certbot-nginx
sudo systemctl enable --now nginx
sudo nginx -t
systemctl is-active nginx
```

6.6 Production filesystem

```
sudo install -d -m 0755 -o root -g root /opt/misty-server
sudo install -d -m 0700 -o root -g root /etc/misty-server
sudo install -d -m 0700 -o root -g root /var/backups/misty/postgres
sudo install -d -m 0750 -o root -g adm  /var/log/misty
```

6.7 Host verification gate

Continue only when all checks below pass:

[ ] A second SSH login using the operator key succeeds.

[ ] Package updates are complete and reboot requirements are understood.

[ ] Only 22, 80, and 443 are intentionally reachable from the Internet.

[ ] Docker and Compose work after a host reboot.

[ ] Nginx config validation succeeds.

[ ] DNS for mistysys.com resolves to the intended host.

7. Build and move an immutable image

7.1 One artifact, many environments

Build the container once from a known source commit. Run tests against that commit, record the image digest, and deploy the same digest. Configuration selects the environment; rebuilding does not.

7.2 Registry path

```
git status --short
git rev-parse HEAD
./test.sh
go vet ./...

docker buildx build   --platform linux/amd64   --tag <registry>/misty-server:<git-sha>   --push .

docker buildx imagetools inspect <registry>/misty-server:<git-sha>
```

If the VPS is arm64, build linux/arm64 or a multi-platform image. In production Compose use image: registry/misty-server@sha256:, not a floating tag.

7.3 No-registry transfer

```
# Build for the VPS architecture.
docker buildx build --platform linux/amd64 --load -t misty-server:<git-sha> .
docker image save misty-server:<git-sha> | gzip > misty-server-<git-sha>.tar.gz
shasum -a 256 misty-server-<git-sha>.tar.gz

# Copy through SSH, verify the checksum on the VPS, then:
gunzip -c misty-server-<git-sha>.tar.gz | sudo docker image load
sudo docker image inspect misty-server:<git-sha>
```

7.4 Artifact record

- Source commit SHA and branch or release tag.
- Image digest, platforms, build timestamp, Go version, and Dockerfile revision.
- Test command results.
- Migration range expected by the release.
- Operator who approved the artifact.

8. Production Compose design

> Current repository note
> deploy/compose.staging.yml is production-shaped but its API uses the database owner account. An owner can bypass row-level security. Use a restricted application role in production.

8.1 Service rules

- PostgreSQL has no published host port and lives only on a private network.
- Migrations use a database owner role and run as a one-shot service before the API.
- The API uses a restricted application role, a read-only filesystem, tmpfs for /tmp, dropped capabilities, and no-new-privileges.
- The API is published only as 127.0.0.1:8081:8080.
- Long-running services use restart: unless-stopped. Do not add a second custom systemd supervisor around Compose containers.
- The Stripe CLI, trycloudflare tunnel, and dev Worker deploy are absent.

8.2 Reference compose.production.yml

```
name: misty-prod

services:
  postgres:
    image: pgvector/pgvector:pg17@sha256:<pin-real-digest>
    restart: unless-stopped
    environment:
      POSTGRES_DB: ${DB_NAME}
      POSTGRES_USER: ${DB_MIGRATION_USER}
      POSTGRES_PASSWORD: ${DB_MIGRATION_PASSWORD}
      MISTY_APP_DB_USER: ${DB_USER}
      MISTY_APP_DB_PASSWORD: ${DB_PASSWORD}
    volumes:
      - postgres-data:/var/lib/postgresql/data
      - ./scripts/docker/postgres-init-app-role.sh:/docker-entrypoint-initdb.d/10-misty-app-role.sh:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U $$POSTGRES_USER -d $$POSTGRES_DB"]
      interval: 5s
      timeout: 5s
      retries: 20
    networks: [database]

  migrate:
    image: <registry>/misty-server@sha256:<pin-release-digest>
    user: "10001:10001"
    restart: "no"
    environment:
      GOOSE_DRIVER: postgres
      GOOSE_DBSTRING: >-
        host=postgres port=5432 user=${DB_MIGRATION_USER}
        password=${DB_MIGRATION_PASSWORD} dbname=${DB_NAME} sslmode=disable
      GOOSE_MIGRATION_DIR: /app/migrations
    entrypoint: ["/usr/local/bin/goose"]
    command: ["up"]
    depends_on:
      postgres:
        condition: service_healthy
    networks: [database]

  database-permissions:
    image: pgvector/pgvector:pg17@sha256:<pin-real-digest>
    restart: "no"
    environment:
      PGHOST: postgres
      PGPORT: "5432"
      PGDATABASE: ${DB_NAME}
      PGUSER: ${DB_MIGRATION_USER}
      PGPASSWORD: ${DB_MIGRATION_PASSWORD}
      MISTY_APP_DB_USER: ${DB_USER}
    volumes:
      - ./scripts/docker/postgres-grant-app-role.sh:/usr/local/bin/misty-grant-app-role:ro
    depends_on:
      migrate:
        condition: service_completed_successfully
    entrypoint: ["/bin/sh", "/usr/local/bin/misty-grant-app-role"]
    networks: [database]

  api:
    image: <registry>/misty-server@sha256:<pin-release-digest>
    restart: unless-stopped
    env_file: /etc/misty-server/production.env
    environment:
      PORT: "8080"
      DB_HOST: postgres
      DB_PORT: "5432"
      DB_MIGRATION_USER: ""
      DB_MIGRATION_PASSWORD: ""
    ports:
      - "127.0.0.1:8081:8080"
```

```
    read_only: true
    tmpfs:
      - /tmp:rw,noexec,nosuid,size=64m
    cap_drop: [ALL]
    security_opt:
      - no-new-privileges:true
    depends_on:
      database-permissions:
        condition: service_completed_successfully
    healthcheck:
      test: ["CMD-SHELL", "curl --fail --silent http://127.0.0.1:8080/health >/dev/null"]
      interval: 10s
      timeout: 3s
      retries: 12
    networks: [edge, database]

volumes:
  postgres-data:
    name: misty-prod-postgres-data

networks:
  edge: {}
  database:
    internal: true
```

The role initializer runs only when PostgreSQL creates a fresh data directory. On an existing volume, create or rotate the restricted role explicitly before the grant job. A named volume is persistent but is not a backup.

8.3 Validate without starting

```
cd /opt/misty-server
sudo docker compose   --env-file /etc/misty-server/production.env   -f deploy/compose.production.yml
config --quiet
```

9. PostgreSQL roles, migrations, and row security

9.1 Role separation

Role	Used by	Allowed
Database owner	Migration and controlled permission jobs only.	DDL, ownership, extension and schema changes.
Application role	misty-server at runtime.	Only required DML and sequence access; never SUPERUSER, BYPASSRLS, or object owner.
Backup role	Backup automation.	Read access sufficient for dumps; carefully scoped and monitored.
Replication role	Physical standby.	LOGIN REPLICATION only, restricted by network and pg_hba.conf.

9.2 Migration rule

1. Take and verify a pre-deploy backup.
2. Confirm the new application can tolerate the current schema.
3. Run migrations exactly once using the owner role.
4. Reconcile grants and ownership so the API remains restricted.
5. Start the new API and verify schema-version health.
6. For destructive migrations, use expand-and-contract across multiple releases.

9.3 Validate the runtime role

```
SELECT current_user;
SELECT rolname, rolsuper, rolbypassrls
FROM pg_roles
WHERE rolname = current_user;

SELECT schemaname, tablename, rowsecurity, forcerowsecurity
FROM pg_tables
WHERE schemaname = 'public'
ORDER BY tablename;
```

> **Owner accounts defeat the intent of RLS**
> The application startup already warns if its role is SUPERUSER or BYPASSRLS. Treat either warning as a failed production deployment.

10. Nginx and TLS

10.1 Preserve the /api prefix

The server's public base is https://mistysys.com/api. In Nginx, proxy_pass must not have a trailing slash in this location; otherwise Nginx rewrites away the matched /api prefix.

```
map $http_upgrade $connection_upgrade {
    default upgrade;
    ''      close;
}

server {
    listen 80;
    listen [::]:80;
    server_name mistysys.com www.mistysys.com;
    return 301 https://mistysys.com$request_uri;
}

server {
    listen 443 ssl http2;
    listen [::]:443 ssl http2;
    server_name mistysys.com;

    # Certbot-managed certificate directives live here.

    location = /api/health {
        proxy_pass http://127.0.0.1:8081;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }

    location ^~ /api/ {
        proxy_pass http://127.0.0.1:8081;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection $connection_upgrade;
        proxy_read_timeout 3600s;
        proxy_send_timeout 3600s;
    }
}
```

10.2 Install or renew the certificate

```
sudo nginx -t
sudo certbot --nginx -d mistysys.com -d www.mistysys.com
sudo certbot renew --dry-run
systemctl status certbot.timer
```

10.3 Validate the edge

```
curl --fail --show-error http://127.0.0.1:8081/health
curl --fail --show-error https://mistysys.com/api/health
curl -I https://mistysys.com/api/health
openssl s_client -connect mistysys.com:443 -servername mistysys.com </dev/null
```

- The public response must be HTTPS and must not redirect to an internal port.
- Secure cookies require the correct X-Forwarded-Proto=https signal.
- WebSocket paths require HTTP/1.1 Upgrade and Connection forwarding.
- Do not expose the API container port on 0.0.0.0.

11. First production deployment

11.1 Preflight gate

[ ] Source commit and image digest are recorded.

[ ] All repository tests, vetting, Worker tests, and container contract checks pass.

[ ] Production environment validation passes without printing secrets.

[ ] Nginx config and certificate are valid.

[ ] PostgreSQL owner and application roles are distinct.

[ ] A backup destination outside the VPS exists.

[ ] Stripe live-mode endpoint is prepared but can remain disabled until API validation.

[ ] Cloudflare production Worker and R2 resources are separate from development.

[ ] A rollback image digest and responsible operator are identified.

11.2 Deploy sequence

```
cd /opt/misty-server

sudo docker compose   --env-file /etc/misty-server/production.env   -f deploy/compose.production.yml
config --quiet

sudo docker compose   --env-file /etc/misty-server/production.env   -f deploy/compose.production.yml pull

# Create and copy an off-host pre-deploy backup before schema changes.
sudo /usr/local/sbin/misty-backup-postgres

sudo docker compose   --env-file /etc/misty-server/production.env   -f deploy/compose.production.yml run -
-rm migrate

sudo docker compose   --env-file /etc/misty-server/production.env   -f deploy/compose.production.yml run -
-rm database-permissions

sudo docker compose   --env-file /etc/misty-server/production.env   -f deploy/compose.production.yml up -
d api postgres --remove-orphans
```

11.3 Verify in layers

```
sudo docker compose -f deploy/compose.production.yml ps
sudo docker compose -f deploy/compose.production.yml logs --tail=200 api postgres
curl --fail http://127.0.0.1:8081/health
curl --fail https://mistysys.com/api/health
```

1. Create a production test account through the real client.
2. Verify authentication, cookies, and one authorized API request.
3. Exercise an R2 upload and retrieval with a disposable object.
4. Send a Stripe test webhook only in the appropriate Stripe mode and confirm delivery.
5. Open a journal collaboration session and verify Worker-to-API communication.
6. Watch API, Nginx, PostgreSQL, Stripe, and Worker errors for at least 15 minutes.

> **Launch gate**
> Do not declare success from a single /health response. The system is ready only after a real authenticated path, storage path, billing event, and collaboration path have been checked.

12. Stripe production setup

12.1 Development versus production

Development	Production
stripe listen creates a temporary signing secret and forwards to the API container.	Register a persistent public HTTPS webhook endpoint in the Stripe Dashboard.
Test-mode keys and products.	Live-mode keys, prices, customer portal, and webhook signing secret.
Path commonly /stripe/webhook inside the dev network.	Public endpoint https://mistysys.com/api/stripe/webhook.

12.2 Production checklist

[ ] Live products and all four required price IDs exist and are distinct.

[ ] Success, cancel, and portal return URLs are absolute HTTPS URLs.

[ ] Webhook endpoint is https://mistysys.com/api/stripe/webhook.

[ ] The endpoint signing secret is stored in STRIPE_WEBHOOK_SECRET.

[ ] Only required event types are subscribed.

[ ] A delivery reaches the endpoint and receives the expected 2xx response.

[ ] Failed deliveries and reconciliation jobs are monitored.

> **Never use the Stripe CLI secret in production**
> The whsec value printed by stripe listen belongs to that local listener session. Production uses the endpoint signing secret shown for the registered Dashboard webhook.

12.3 Incident actions

- For 404, verify STRIPE_WEBHOOK_PATH and that Nginx preserves /api.

- For signature errors, verify the live endpoint secret, raw request-body handling, and mode.
- For 5xx, preserve the Stripe event ID and application request ID; fix the cause, then retry safely.
- Do not manually apply billing state without idempotency and reconciliation evidence.

13. Cloudflare Worker, Durable Objects, and R2

13.1 Production Worker

The journal collaboration Worker is deployed separately from the API container. Its production configuration should call https://mistysys.com/api directly. The development tunnel and automatic dev deployment service do not belong on the VPS.

1. Select the production Cloudflare account and Worker name.
2. Verify the production variable MISTY_INTERNAL_API_BASE=https://mistysys.com/api.
3. Create scoped Worker and R2 credentials; never reuse the broad development token.
4. Set Worker secrets through Wrangler or the Cloudflare secret interface, not source control.
5. Run Worker type checks and Vitest coverage locally.
6. Deploy the production Worker explicitly and record its version.
7. Verify a real collaboration ticket and WebSocket session end to end.

13.2 R2

- Use a dedicated production bucket and least-privilege credentials.
- Apply the required CORS policy explicitly and review it before using misty-cli server r2 configure-cors --apply.
- Keep bucket names, endpoints, and keys environment-specific.
- Inventory object count and total bytes. Alert on unexpected drops or growth.
- Maintain a separate recovery copy or versioning strategy appropriate to deletion risk.

13.3 Durable Object recovery

Cloudflare SQLite-backed Durable Objects provide point-in-time recovery for a rolling historical window. Document how to identify affected objects, create recovery bookmarks when appropriate, restore safely, and verify application reconciliation. This is independent from PostgreSQL.

> **Cloudflare state is still your operational responsibility**
> Provider durability protects hardware failures. It is not a substitute for retention policy, access control, deletion recovery, and a tested operator procedure.

14. Backup system

14.1 Starting policy

Layer	Frequency	Retention	Location
PostgreSQL logical dump	Nightly and before every migration.	14 daily, 8 weekly, 12 monthly.	Encrypted off-host object storage plus local short-term cache.
PostgreSQL WAL/PITR	Continuous after configured.	Enough for the selected RPO and restore window.	Separate durable archive.
R2 objects	Continuous replication or daily inventory/copy.	Based on customer deletion and legal policy.	Separate bucket/account/provider where feasible.
Configuration	After every change.	Current plus history.	Encrypted vault and off-host recovery package.
Durable Objects	Provider PITR plus documented bookmarks/runbook.	Cloudflare window.	Cloudflare control plane and incident record.

14.2 Logical PostgreSQL backup

```
stamp="$(date -u +%Y%m%dT%H%M%SZ)"
backup_dir="/var/backups/misty/postgres/${stamp}"
sudo install -d -m 0700 "$backup_dir"

# Run with a restricted backup credential supplied without exposing it in process output.
pg_dump --host 127.0.0.1 --port <private-or-tunneled-port> --username <backup-role> --dbname
misty --format custom --compress 9 --file "$backup_dir/misty.dump"

pg_dumpall --host 127.0.0.1 --port <private-or-tunneled-port> --username <backup-role> --globals-
only > "$backup_dir/globals.sql"

sha256sum "$backup_dir/misty.dump" "$backup_dir/globals.sql" > "$backup_dir/SHA256SUMS"
```

If PostgreSQL is not published on the host, execute pg_dump in a temporary container on the database network or stream it through docker compose exec. Avoid placing passwords directly in shell history. Custom format supports selective and parallel pg_restore.

14.3 Backup job must fail loudly

- Exit nonzero on any failed command.
- Write backup timestamp, database version, schema migration version, source host, and checksum.
- Copy off-host before reporting success.
- Alert when the newest verified backup exceeds the allowed age.
- Never delete the last known-good copy during retention cleanup.
- Test the produced archive with pg_restore --list and a real isolated restore.

14.4 Why not tar a live database volume

A blind filesystem copy of a running PostgreSQL data directory can be inconsistent. Use pg_dump for logical backups or pg_basebackup plus WAL for physical recovery. Provider snapshots are useful only when their database consistency and restore behavior are understood and tested.

14.5 3-2-1 evidence

[ ] Three copies exist: live data plus two recoverable copies.

[ ] Two different storage systems or media are involved.

[ ] At least one copy is off-host and isolated from routine application credentials.

[ ] Checksums are validated after transfer.

[ ] A dated restore drill proves the backup is usable.

15. Restore a PostgreSQL backup

> **Restore into a new target first**
> Do not overwrite the only production database during diagnosis. Preserve the old volume or host, restore into an isolated target, validate, then switch traffic deliberately.

15.1 Standard restore sequence

1. Declare the incident and stop writes if continued writes would worsen inconsistency.
2. Record the incident time, suspected bad interval, source backup, and intended restore point.
3. Provision a clean PostgreSQL instance with a compatible major version and required extensions.
4. Restore global roles carefully; do not overwrite unrelated roles on a shared cluster.
5. Create the target database owned by the migration owner.
6. Restore the custom-format dump with pg_restore.
7. Run required migrations only if the restored schema is older than the chosen application.
8. Reconcile grants and verify that the API role is not owner, superuser, or BYPASSRLS.
9. Run integrity, row-count, application, R2-reference, and billing reconciliation checks.
10. Switch the API database endpoint, restart the API, and monitor.
11. Keep the old database read-only until the recovery is accepted.

15.2 Example

```
createdb --host <restore-host> --username <owner-role>   --owner <owner-role> misty_restore

pg_restore   --host <restore-host>   --username <owner-role>   --dbname misty_restore   --clean --if-
exists   --no-owner   --jobs 4   /secure/path/misty.dump

PGHOST=<restore-host> PGDATABASE=misty_restore PGUSER=<owner-role> MISTY_APP_DB_USER=misty_app   /bin/sh
scripts/docker/postgres-grant-app-role.sh
```

15.3 Validation queries

```
SELECT now(), current_database(), current_user;
SELECT extname, extversion FROM pg_extension ORDER BY extname;
SELECT version_id, is_applied FROM goose_db_version ORDER BY id DESC LIMIT 5;
SELECT pg_size_pretty(pg_database_size(current_database()));
```

Also compare business-level counts and recent records, sample referenced R2 objects, verify authentication, and run Stripe reconciliation. A database that starts is not necessarily a correct restore.

16. Copy production-like data to development

16.1 Prefer synthetic fixtures

The safest development database is generated from realistic fixtures. Use a production snapshot only when a bug or performance problem cannot be reproduced otherwise.

16.2 Controlled clone flow

1. Approve a specific need, scope, owner, and expiration date.
2. Create a logical dump using a restricted role.
3. Restore into an isolated scrub environment that cannot contact production integrations.
4. Remove or irreversibly transform personal data, authentication material, sessions, tokens, billing identifiers, message content, and private assets.
5. Replace emails and phone numbers with deterministic fake values when relationship testing requires stable joins.
6. Delete secrets, OAuth credentials, API keys, password reset tokens, webhook deliveries, and provider refresh tokens.
7. Point all URLs and provider configurations to development or disabled sinks.
8. Validate the scrub with automated forbidden-pattern and sample-record checks.
9. Create the developer snapshot only after validation; encrypt it and set an expiration.
10. Destroy the raw dump and scrub environment under the approved retention procedure.

> **Never merely change the database hostname**
> A cloned database may contain credentials or queued work that calls real customers and providers. Network isolation and secret removal must happen before any application starts against it.

16.3 Environment clone checklist

[ ] No production passwords, sessions, tokens, or encryption keys remain.

[ ] No live Stripe customer, subscription, payment, or webhook credentials remain.

[ ] No live email, Slack, Discord, Google, Notion, or other OAuth credentials remain.

[ ] Personally identifying content is removed or irreversibly transformed.

[ ] R2 references point only to a development bucket or fixtures.

[ ] Background jobs and outbound notifications are disabled until verified.

[ ] The snapshot has an owner, checksum, creation date, and expiration date.

17. PostgreSQL streaming standby

A physical streaming standby continuously replays WAL from the primary and can reduce recovery time after a host failure. It must run the same PostgreSQL major version and compatible extensions. Place it on a different failure domain and private network.

> **Replication is not backup**
> Accidental deletes, bad migrations, and corruption can replicate immediately. Keep independent historical backups and PITR even after a standby exists.

17.1 Network and host prerequisites

- Separate VPS, disk, and ideally provider zone.
- Private WireGuard/Tailscale or provider network; never public PostgreSQL.
- Same PostgreSQL major version, image digest, extensions, locale, and compatible configuration.
- Time synchronization, monitored disk capacity, and sufficient bandwidth.
- A tested procedure to rebuild the standby from the primary.

17.2 Primary parameters

```
wal_level = replica
max_wal_senders = 10
max_replication_slots = 10
wal_keep_size = 2048MB
hot_standby = on
password_encryption = scram-sha-256
```

Tune values to workload and RPO. PostgreSQL 17 can use a replication slot with controlled WAL retention. Without monitoring, a disconnected slot can retain WAL until the primary disk fills.

17.3 Replication role and network rule

```
CREATE ROLE misty_replica
  WITH LOGIN REPLICATION
  PASSWORD '<long-random-replication-password>';

# pg_hba.conf: allow only the standby's private address.
hostssl replication misty_replica <standby-private-ip>/32 scram-sha-256
```

17.4 Create a physical slot

```
SELECT * FROM pg_create_physical_replication_slot('misty_standby_1');
SELECT slot_name, slot_type, active, restart_lsn
FROM pg_replication_slots;
```

17.5 Seed the standby

Stop the empty standby, ensure its target data directory is empty, and run pg_basebackup from the standby host. The -R flag writes standby connection settings; -X stream includes WAL.

```
pg_basebackup   --host <primary-private-ip>   --port 5432   --username misty_replica   --pgdata <empty-
standby-data-directory>   --format plain   --wal-method stream   --write-recovery-conf   --slot
misty_standby_1   --checkpoint fast   --progress   --manifest-checksums SHA256
```

17.6 Verify replication

```
# On primary:
SELECT application_name, client_addr, state, sync_state,
       sent_lsn, write_lsn, flush_lsn, replay_lsn
FROM pg_stat_replication;

# On standby:
SELECT pg_is_in_recovery(),
       pg_last_wal_receive_lsn(),
       pg_last_wal_replay_lsn(),
       now() - pg_last_xact_replay_timestamp() AS replay_delay;
```

17.7 Alert conditions

- Standby disconnected or pg_stat_replication row absent.
- Replay lag exceeds the chosen RPO threshold.
- Replication slot retained WAL grows unexpectedly.
- Primary or standby disk exceeds 70 percent, then critical at 85 percent.
- Timeline or WAL receiver errors appear.
- Read-only standby unexpectedly accepts writes or is no longer in recovery.

18. Manual failover and rejoin

18.1 Fence before promotion

Never promote a standby while the old primary may still accept writes. That creates split brain. First fence the old primary by stopping PostgreSQL, revoking its network, or powering it off with confirmed control-plane evidence.

18.2 Failover sequence

1. Declare the incident and freeze deployments.
2. Determine whether the primary is truly unavailable and record the last confirmed transaction time.
3. Fence the old primary from clients and the standby.
4. Check standby replay lag and decide whether the potential data loss is acceptable.
5. Promote the standby using SELECT pg_promote() or pg_ctl promote.
6. Verify pg_is_in_recovery() returns false and the new primary accepts controlled writes.
7. Update the API database endpoint or stable private DNS name.
8. Restart API containers and run health plus business-flow checks.
9. Monitor errors, missing transactions, billing, and queues.
10. Rebuild the old primary as a new standby. Do not simply start it against the new primary.

18.3 Promotion commands

```
# On the standby after fencing the old primary:
psql --dbname postgres --command "SELECT pg_promote(wait_seconds => 60);"
psql --dbname postgres --command "SELECT pg_is_in_recovery();"
```

> **No automatic failback**
> Once the standby is promoted, it is the primary. Restore redundancy by taking a fresh base backup or using a deliberate rewind procedure with verified prerequisites.

19. Deploy updates and roll back

19.1 Routine release

1. Run the complete local verification suite and record the source commit.
2. Build and publish an immutable image; record its digest.
3. Review migrations for locks, runtime, reversibility, and backward compatibility.
4. Take a verified off-host pre-deploy database backup.
5. Update only the pinned image digest in production configuration.
6. Validate Compose, pull the image, run migrations, reconcile permissions, and recreate API.
7. Run layered health and canary checks; inspect logs and metrics.
8. Record release time, operator, digest, migration version, and result.

19.2 Application rollback

```
# Change the API image back to the recorded prior digest.
sudo docker compose   --env-file /etc/misty-server/production.env   -f deploy/compose.production.yml up -
d --no-deps api

curl --fail http://127.0.0.1:8081/health
curl --fail https://mistysys.com/api/health
```

19.3 Migration compatibility

- Prefer additive schema changes first: add columns/tables/indexes without removing old behavior.
- Deploy code that can use both old and new shapes.
- Backfill in bounded, observable jobs.
- Switch reads and writes only after compatibility is proven.
- Remove old schema in a later release after rollback windows close.

> **An image rollback cannot undo a destructive migration**
> If the previous application cannot run on the new schema, recovery may require a database restore and a coordinated data reconciliation. Review this before every migration.

20. Monitoring and alerting

20.1 Minimum signals

Layer	Monitor
Edge	HTTPS availability, certificate expiry, Nginx 4xx/5xx, upstream timeouts, request latency.
API	Container health, restart count, /health, error rate, latency, panic/fatal logs, worker health.
PostgreSQL	Availability, connections, long transactions, locks, slow queries, disk, checkpoints, WAL, schema version.
Backups	Newest successful off-host backup age, size anomalies, checksum result, restore-drill age.
Replica	Connection state, replay lag, retained WAL, disk, recovery state.
Stripe	Webhook delivery failures, signature failures, reconciliation lag.
Cloudflare	Worker exceptions, Durable Object errors, R2 errors, object-count anomalies, API latency.
Host	CPU, memory, swap, disk, inode use, load, clock, security updates, Docker daemon.

20.2 Starter thresholds

- Page when public health fails for 2-5 consecutive minutes.
- Warn at 70 percent disk and page at 85 percent.
- Page if no verified off-host database backup exists within the RPO window.
- Warn when replication lag approaches the RPO; page when it exceeds it.
- Warn 30 days before TLS certificate expiry and page at 7 days.
- Page on sustained 5xx rate or sharp latency regression, not a single expected error.

20.3 Logs

- Keep Nginx access/error logs and API structured logs with request IDs.
- Do not log passwords, tokens, cookies, authorization headers, webhook secrets, or full sensitive payloads.
- Set rotation and retention so logs cannot fill the host.
- Ship critical logs off-host if the VPS is a single failure domain.
- Synchronize UTC timestamps across host, containers, Stripe, and Cloudflare.

21. Routine operations cadence

Daily

[ ] Public and local health checks are green.

[ ] No unexplained container restarts or sustained 5xx errors.

[ ] Latest PostgreSQL backup is off-host and within RPO.

[ ] Disk, WAL, and replication lag are normal.

[ ] Stripe and Cloudflare error queues are clear or assigned.

Weekly

[ ] Review host, Docker, Nginx, API, database, and Worker errors.

[ ] Review security updates and schedule reboots.

[ ] Verify backup checksums and run an automated isolated restore.

[ ] Review failed jobs, billing reconciliation, and R2 inventory changes.

[ ] Remove unused images only after verifying rollback artifacts remain available.

Monthly

[ ] Perform a human-observed restore drill and record measured RTO.

[ ] Review account access, SSH keys, Docker group membership, Cloudflare tokens, and Stripe users.

[ ] Review secret ages and rotation schedule.

[ ] Test certificate renewal and expiration alerting.

[ ] Review database growth, slow queries, indexes, and capacity forecast.

[ ] If replication exists, rehearse promotion in a non-production environment.

Quarterly

[ ] Run a full disaster-recovery exercise from off-host backups.

[ ] Review RPO/RTO with actual business needs.

[ ] Review dependency, OS, PostgreSQL, Docker, Nginx, Go, and Worker runtime upgrades.

[ ] Test operator access when the usual laptop, DNS, or primary VPS is unavailable.

[ ] Update this manual from incident and drill findings.

22. Incident runbooks

22.1 Public API returns 503

1. Check public and loopback health separately.
2. If loopback works, inspect Nginx upstream, route prefix, headers, and certificate.
3. If loopback fails, inspect API container health and logs.
4. Check PostgreSQL health, credentials, connectivity, migrations, locks, and disk.
5. Check required external configuration validation.
6. Do not restart repeatedly without preserving the first failure evidence.

```
curl -v https://mistysys.com/api/health
curl -v http://127.0.0.1:8081/health
sudo nginx -t
sudo journalctl -u nginx --since "-15 minutes"
sudo docker compose -f deploy/compose.production.yml ps
sudo docker compose -f deploy/compose.production.yml logs --since=15m api postgres
```

22.2 Disk is nearly full

1. Stop nonessential writes and determine which filesystem and directory grew.
2. Check PostgreSQL data, WAL, logs, Docker layers, and backup cache separately.
3. If a replication slot retained WAL, fix or remove only the confirmed obsolete slot.
4. Move verified backups off-host before removing local cache.
5. Expand storage or restore normal retention with evidence; do not delete live database files.

22.3 Failed migration

- Keep the old API version serving if it remains schema-compatible.
- Capture migration output, database locks, schema version, and pre-deploy backup identity.
- Do not mark a migration successful manually without understanding its partial effects.
- Choose forward repair, tested down migration, or restore based on data changes and compatibility.

22.4 Suspected secret leak

1. Preserve evidence without spreading the secret.
2. Revoke or rotate the affected credential from a trusted device.
3. Update production, recreate affected services, and verify.
4. Review provider audit logs, database access, Stripe events, Cloudflare changes, and Git history.
5. Rotate dependent secrets if the exposed credential could reveal them.
6. Document scope, impact, timeline, and preventive action.

22.5 Bad release

- Freeze deployments and record the failing digest.
- If schema-compatible, roll API back to the prior digest.
- If data or schema changed incompatibly, follow the database recovery decision tree.
- Verify billing, jobs, R2 operations, and collaboration state after recovery.

23. Command quick reference

Development

```
misty-cli doctor
misty-cli check all
misty-cli server up
misty-cli server up --detach
misty-cli server up --no-build
misty-cli server logs
misty-cli server down
misty-cli server down --volumes   # destructive: deletes dev data
```

Production status

```
sudo docker compose -f deploy/compose.production.yml ps
sudo docker compose -f deploy/compose.production.yml logs -f --tail=200 api
curl --fail http://127.0.0.1:8081/health
curl --fail https://mistysys.com/api/health
sudo nginx -t
sudo systemctl status docker nginx
df -h
```

PostgreSQL

```
SELECT version();
SELECT current_database(), current_user;
SELECT pg_is_in_recovery();
SELECT * FROM pg_stat_replication;
SELECT * FROM pg_replication_slots;
SELECT pg_size_pretty(pg_database_size(current_database()));
```

Artifact and release identity

```
git rev-parse HEAD
docker image inspect <image-ref> --format '{{json .RepoDigests}}'
docker compose -f deploy/compose.production.yml config --images
```

Remote Docker without opening the daemon port

```
docker context create misty-prod --docker "host=ssh://mistyops@<vps>"
docker --context misty-prod ps
```

24. First-launch master checklist

Host and edge

[ ] SSH key access and provider console recovery are tested.

[ ] OS is patched; time is UTC and synchronized.

[ ] Only intended ports are reachable.

[ ] Docker starts after reboot.

[ ] Nginx preserves /api and forwards proxy/WebSocket headers.

[ ] TLS certificate and automatic renewal are verified.

Application and configuration

[ ] Production image is pinned by digest and matches the tested source commit.

[ ] MISTY_ENVIRONMENT=production validation passes.

[ ] API is bound only to 127.0.0.1:8081.

[ ] Production origins and HTTPS URLs are correct.

[ ] Secrets are root-only, off Git, and backed up encrypted.

[ ] Application database role is restricted and does not bypass RLS.

External systems

[ ] Stripe uses live products, prices, keys, and the Dashboard webhook secret.

[ ] Stripe endpoint is https://mistysys.com/api/stripe/webhook.

[ ] Production Worker points to https://mistysys.com/api and has scoped secrets.

[ ] Production R2 bucket, endpoint, CORS, and least-privilege credentials are verified.

[ ] A real collaboration and object-storage path succeeds.

Recovery

[ ] A pre-launch logical dump is copied off-host and checksum-verified.

[ ] An isolated PostgreSQL restore succeeds.

[ ] R2 and Durable Object recovery responsibilities are documented.

[ ] Rollback image digest and procedure are recorded.

[ ] Monitoring covers health, disk, backups, replication, Stripe, Cloudflare, and TLS.

[ ] The incident owner and communication path are known.

> Production-ready means recoverable
> The first launch is complete only when the service works and an operator can prove how to restore it.

25. Recommended maturity roadmap

Stage	Add	Exit evidence
1. Single node	VPS, Nginx, Compose, private PostgreSQL, immutable API image.	End-to-end production checks pass; no public DB/API backend ports.
2. Recoverable	Nightly logical backups, off-host copy, R2/DO plan, alerts.	A clean-host restore drill meets the initial RTO.
3. Point in time	WAL archiving and PITR procedure.	Restore to a chosen timestamp is rehearsed.
4. Standby	Second failure domain, streaming replica, lag and slot alerts.	Controlled failover and rejoin are rehearsed.
5. Hardened	Central logs, secret manager, vulnerability cadence, access reviews.	Quarterly recovery and security drills produce tracked actions.
6. Automated HA	Only if business needs justify orchestration complexity.	Split-brain prevention and failure modes are proven under test.

Do not skip directly to automatic high availability. A simple system with tested backups and a clear manual failover is safer than an untested cluster that can make destructive decisions automatically.

Appendix A. Environment inventory

The following inventory is the production-critical subset discovered from the current server validation and deployment flow. Feature providers may add more variables. Generate a canonical inventory from source during each release and diff it against the production secret store.

Variable	Purpose / rule
MISTY_ENVIRONMENT	Set to production to activate strict validation.
PORT	8080 inside the container.
MISTY_PUBLIC_API_URL	https://mistysys.com/api.
MISTY_ALLOWED_ORIGINS	Comma-separated exact origins; no wildcard in production.
TRUST_PROXY_HEADERS	true behind loopback Nginx.
DB_HOST, DB_PORT, DB_NAME	Private PostgreSQL endpoint and database.
DB_USER, DB_PASSWORD	Restricted runtime application role.
DB_MIGRATION_USER, DB_MIGRATION_PASSWORD	Compose/deployment inputs for migration only; explicitly clear them in the API service.
R2_ENDPOINT, R2_BUCKET	Production R2 location.
R2_ACCESS_KEY, R2_SECRET_KEY	Least-privilege production object access.
SPACE_LINK_ENCRYPTION_KEY	Production encryption input; rotation needs a data compatibility plan.
STRIPE_SECRET_KEY	Stripe live secret key.
STRIPE_WEBHOOK_SECRET	Dashboard endpoint signing secret, not stripe listen output.
STRIPE_WEBHOOK_PATH	/api/stripe/webhook for current public routing.
STRIPE_PRICE_PRO_MONTHLY	Distinct live price ID.
STRIPE_PRICE_PRO_YEARLY	Distinct live price ID.
STRIPE_PRICE_MAX_MONTHLY	Distinct live price ID.
STRIPE_PRICE_MAX_YEARLY	Distinct live price ID.
STRIPE_CHECKOUT_SUCCESS_URL	Absolute HTTPS client URL.
STRIPE_CHECKOUT_CANCEL_URL	Absolute HTTPS client URL.
STRIPE_PORTAL_RETURN_URL	Absolute HTTPS client URL.

Appendix B. Suggested production layout

```
/opt/misty-server/
  deploy/
    compose.production.yml
  scripts/
    docker/
      postgres-init-app-role.sh
      postgres-grant-app-role.sh
  releases/
    <release-manifest>.json

/etc/misty-server/
  production.env              # root:root 0600
  backup.env                  # root:root 0600, if required

/var/backups/misty/
  postgres/                   # short local cache only

/var/log/misty/
  backup.log
  restore-drills.log
```

- Keep Compose and deployment scripts versioned and reviewable.
- Keep secrets outside /opt and outside source control.
- Keep enough local backup cache for diagnosis, but require verified off-host transfer.
- Record each deployment and restore drill in an append-only operational log or ticket system.

Appendix C. Authoritative references

These references support the operational patterns in this manual. Recheck them before major platform upgrades because product behavior and recommended procedures change.

Docker

- Docker Engine: Install on Ubuntu
- Docker Compose: Use Compose in production
- Docker Compose: Control startup and shutdown order
- Docker Engine: Start containers automatically
- Docker Engine security
- Protect the Docker daemon socket
- Docker volumes

Nginx and TLS

- Nginx HTTP proxy module
- Certbot documentation

PostgreSQL 17

- SQL dump backup and restore

- pg_dump
- pg_restore
- Continuous archiving and point-in-time recovery
- pg_basebackup
- High availability, load balancing, and replication
- Hot standby
- pg_replication_slots

Stripe and Cloudflare

- Stripe webhooks
- Cloudflare Durable Objects SQLite storage and point-in-time recovery
- Cloudflare R2 durability
- Cloudflare R2 bucket creation
- Cloudflare R2 object lifecycles

Repository-specific sources reviewed

- Dockerfile
- docker-compose.yml
- deploy/compose.staging.yml
- internal/app/production_environment.go
- internal/platform/config
- internal/platform/postgres
- cloudflare/journal-collab/wrangler.jsonc

> **Keep this manual alive**
> Update it whenever a route, environment variable, provider, data store, migration strategy, deployment target, or recovery objective changes.